

Diseño y Simulación de una Trayectoria de Datos Microcodificada de 4 bits

Josafat Vásquez (9-764-2024)
Facultad de Ingeniería Eléctrica
Universidad Tecnológica de Panamá
Ciudad de Panamá, Panamá
josafat.vasquez@utp.com.pa

Resumen— En este proyecto se diseñó y simuló una trayectoria de datos microcodificada de 4 bits en Verilog. El sistema integra registros, un multiplexor, una ALU, registros de banderas y una unidad de control basada en microcódigo. La simulación en ModelSim permitió verificar el avance de la dirección de microprograma, la carga selectiva de registros y las bifurcaciones por banderas.

Palabras clave— microcódigo, datapath, Verilog, ModelSim, ALU, unidad de control, banderas, microstore.

I. INTRODUCCIÓN

Una trayectoria de datos es el conjunto de registros, buses y bloques combinacionales que permiten mover y transformar información dentro de un procesador. En un sistema microcodificado, las señales que controlan dicha trayectoria no se generan directamente desde una máquina de estados extensa, sino desde una memoria de control que almacena microinstrucciones.

El objetivo de este proyecto fue implementar un procesador didáctico de 4 bits capaz de ejecutar un pequeño programa de demostración. El diseño se separó en módulos para facilitar su verificación: registros, multiplexor, ALU, banderas, datapath completo, microstore, unidad de control y banco de pruebas.

II. TRAYECTORIA DE DATOS

La trayectoria de datos contiene tres registros principales, identificados como A, B y C. El registro C alimenta la entrada X de la ALU, mientras que la entrada Y se obtiene desde un multiplexor de cuatro entradas. El resultado de la ALU, denominado Z, regresa a los registros A, B y C; sin embargo, cada registro se actualiza únicamente cuando su señal de habilitación está activa.

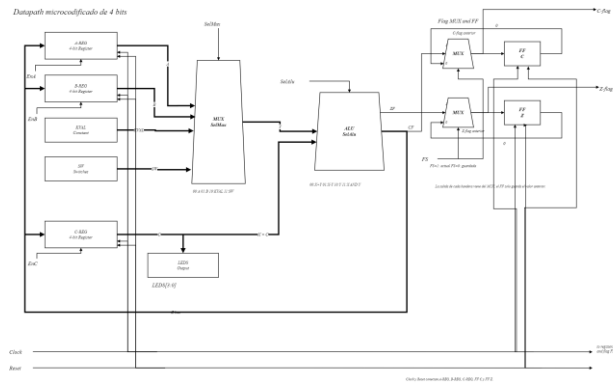


Figura 1. Trayectoria de datos microcodificada de 4 bits.

SelMux	Entrada Y
00	A
01	B
10	KVAL
11	SW

SelAlu	Operación
00	$Z = X + Y$
01	$Z = X - Y$

10	$Z = Y$
11	$Z = X \text{ AND } Y$

Las banderas C y Z se generan a partir de la salida de la ALU. La bandera Z indica resultado cero, mientras que la bandera C se usa como acarreo en suma y como indicador de préstamo en resta. La señal FS permite seleccionar si las banderas visibles corresponden a la operación actual o al valor previamente almacenado.

III. UNIDAD DE CONTROL

La unidad de control es microprogramada. Su contador de dirección produce ADDx, que selecciona una palabra dentro del microstore. Esa palabra contiene señales para el datapath y campos de secuencia que determinan si la ejecución continúa con la siguiente dirección o si se carga una dirección de salto.

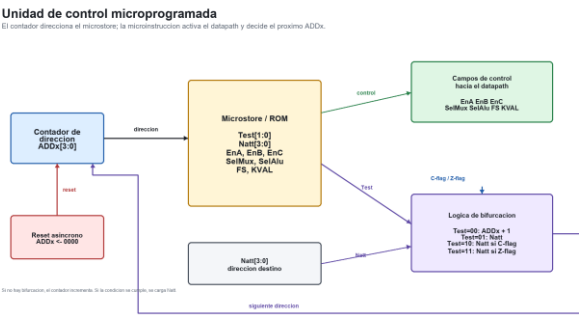


Figura 2. Unidad de control microprogramada.

Campo	Descripción
ADDx	Dirección actual del microprograma.
TEST	Selecciona la condición de bifurcación.
NATT	Dirección destino si se toma el salto.
EnA/EnB/EnC	Habilitan carga de registros.
SelMux/SelAlu	Seleccionan fuente de datos y operación.
FS	Controla la selección de banderas.

TEST	Condición
00	No salto; ADDx+1.
01	Salto incondicional.
10	Salta si C-flag=1.
11	Salta si Z-flag=1.

IV. MICROCÓDIGO SIMBÓLICO

El microcódigo simbólico describe las operaciones de alto nivel que se almacenan posteriormente como campos binarios dentro del microstore. Cada línea corresponde a una dirección de control y se traduce en enables, selectores de MUX, operación de ALU, constante KVAL y condición de salto.

Lista simbólica de demostración

```
0000 Start: C <- F
0001 GetSW: A, B <- SW
0010      B, C <- C - A; if Z-flag* goto Start
0011      C <- A
0100 Top:  C <- C + 1
0101      C - B; if C-flag goto Top
0110      goto Start
```

La marca Z-flag* indica que la decisión utiliza una bandera almacenada previamente. En la demostración, el testbench inicia con SW=0000 para comprobar el retorno a Start y luego cambia a SW=0100 para observar el recorrido normal del programa.

Carta ASM - lectura y decision inicial

Rectángulos: operaciones de estado. Diamantes: decisiones de bifurcación.

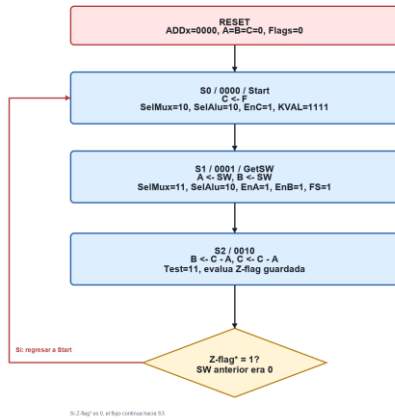
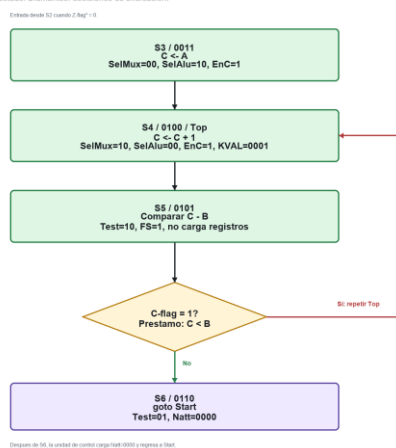


Figura 3. Carta ASM: lectura y decisión inicial.

Carta ASM - lazo de conteo

Rectángulos: operaciones de estado. Diamantes: decisiones de bifurcación.



Después de S6, la unidad de control carga Start=0000 y regresa a Start.

Figura 4. Carta ASM: lazo de conteo.

V. CONTENIDO DEL MICROSTORE

El microstore implementa una ROM combinacional indexada por ADDx. Las direcciones no utilizadas se redirigen al inicio para mantener el sistema en una trayectoria de ejecución definida.

Dir	TEST	NATT	En	SM	SA	FS	KVAL	Operación
0000	00	0000	001	10	10	0	1111	C <- F
0001	00	0000	110	11	10	1	0000	A,B <- SW
0010	11	0000	011	00	01	0	0000	B,C <- C-A; if Z*
0011	00	0000	001	00	10	0	0000	C <- A
0100	00	0000	001	10	00	0	0001	C <- C+1
0101	10	0100	000	01	01	1	0000	C-B; if C
0110	01	0000	000	00	00	0	0000	goto Start

Convención: En = EnA EnB EnC, SM = SelMux y SA = SelAlu.

VI. TABLA DE ACONTECIMIENTOS

Para hacer la tabla de acontecimientos más legible se agruparon los ciclos por función. Los valores A, B y C representan el contenido observado después del flanco activo del reloj; ADDx representa la dirección de microinstrucción visible durante el ciclo.

Ciclo	ADDx	Evento	Resultado observado
0-1	0000	Reset activo	A=B=C=0; banderas en 0.
2	0001	Carga inicial	C=F desde KVAL.
3	0010	SW=0	Z-flag* provoca retorno a Start.
6	0010	SW=4	A=4, B=4; continúa ejecución.
7-8	0011/0100	Preparación	C toma A y luego se incrementa.
9-21	0101/0100	Lazo Top	C aumenta mientras C-flag=1.
22-23	0110/0000	Fin de vuelta	Salto incondicional a Start.

VII. CONCLUSIONES GENERALES

El proyecto permitió comprobar el funcionamiento de una arquitectura microcodificada sencilla. La separación entre datapath y unidad de control facilita identificar qué bloque procesa datos y qué bloque genera las señales de gobierno.

La simulación en ModelSim verificó que el microstore produce las señales esperadas, que los registros sólo cargan cuando sus enables están activos y que las bifurcaciones dependen correctamente de TEST, NATT y las banderas C/Z.

El uso de banderas guardadas mediante FS fue un punto importante del diseño, ya que permitió demostrar la diferencia entre una bandera generada en el ciclo actual y una bandera retenida para una decisión posterior.

REFERENCIAS

- [1] S. Palnitkar, Verilog HDL: A Guide to Digital Design and Synthesis, 2nd ed. Prentice Hall, 2003.
- [2] Siemens EDA, ModelSim Intel FPGA Edition User Manual.
- [3] GOWIN Semiconductor, Tang Nano 9K FPGA Development Board Documentation.

APÉNDICE A. RESULTADOS ACTUALES DE SIMULACIÓN

Las siguientes figuras muestran la corrida actual de simulación en ModelSim. La consola presenta la tabla de valores por ciclo y la ventana Wave muestra señales de entrada, registros, banderas y control microcodificado.

```
VSIM65> run -all
```

#	Tiempo	Ciclo	Reset	SW	Direccion	REG-A	REG-B	REG-C	Z-flag	C-flag	Branch
#	6000	0	1	0000	0000	0000	0000	0000	0	0	0
#	16000	1	1	0000	0000	0000	0000	0000	0	0	0
#	26000	2	0	0000	0001	0000	0000	1111	1	0	0
#	36000	3	0	0000	0010	0000	0000	1111	1	0	1
#	46000	4	0	0000	0000	0000	1111	1111	1	0	0
#	56000	5	0	0000	0001	0000	1111	1111	1	0	0
#	66000	6	0	0100	0010	0100	0100	1111	0	0	0
#	76000	7	0	0100	0011	0100	1011	1011	0	0	0
#	86000	8	0	0100	0100	0100	1011	0100	0	0	0
#	96000	9	0	0100	0101	0100	1011	0101	0	1	1
#	106000	10	0	0100	0100	0100	1011	0101	0	1	0
#	116000	11	0	0100	0101	0100	1011	0110	0	1	1
#	126000	12	0	0100	0100	0100	1011	0110	0	1	0
#	136000	13	0	0100	0101	0100	1011	0111	0	1	1
#	146000	14	0	0100	0100	0100	1011	0111	0	1	0
#	156000	15	0	0100	0101	0100	1011	1000	0	1	1
#	166000	16	0	0100	0100	0100	1011	1000	0	1	0
#	176000	17	0	0100	0101	0100	1011	1001	0	1	1
#	186000	18	0	0100	0100	0100	1011	1001	0	1	0
#	196000	19	0	0100	0101	0100	1011	1010	0	1	1
#	206000	20	0	0100	0100	0100	1011	1010	0	1	0
#	216000	21	0	0100	0101	0100	1011	1011	1	0	0
#	226000	22	0	0100	0110	0100	1011	1011	1	0	1
#	236000	23	0	0100	0000	0100	1011	1011	1	0	0
#	246000	24	0	0100	0001	0100	1011	1111	0	0	0
#	256000	25	0	0100	0010	0100	0100	1111	0	0	0
#	266000	26	0	0100	0011	0100	1011	1011	0	0	0
#	276000	27	0	0100	0100	0100	1011	0100	0	0	0
#	286000	28	0	0100	0101	0100	1011	0101	0	1	1
#	296000	29	0	0100	0100	0100	1011	0101	0	1	0
#	306000	30	0	0100	0101	0100	1011	0110	0	1	1
#	316000	31	0	0100	0100	0100	1011	0110	0	1	0

Figura A1. Salida de consola de ModelSim.

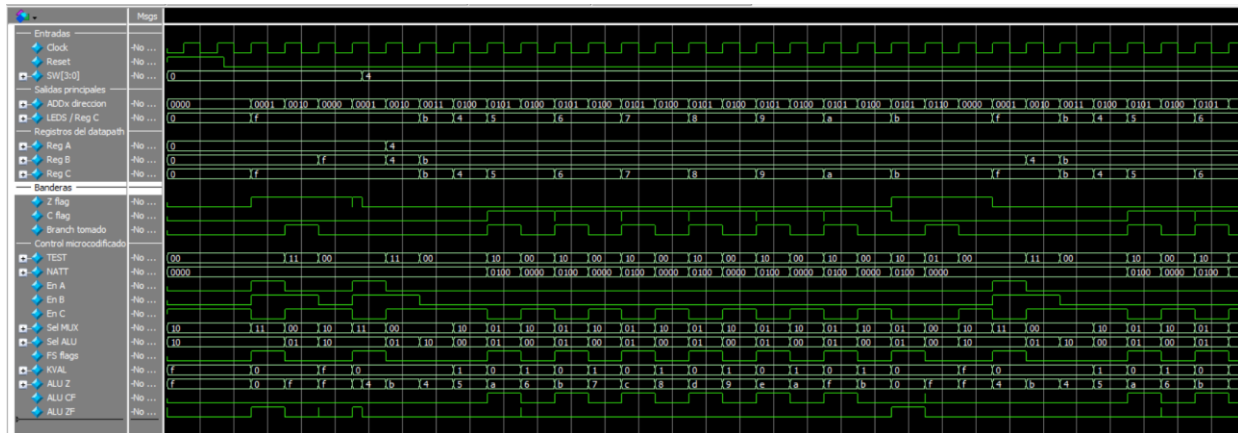


Figura A2. Formas de onda de la simulación.

APÉNDICE B. CÓDIGO FUENTE VERILOG

La Tabla B1 lista los módulos Verilog del diseño y el archivo de simulación. En las páginas siguientes se presenta el código fuente RTL actual de forma compacta. Cada archivo RTL incluye un bloque de comentario con nombre del archivo, autor, fecha y descripción breve.

Archivo	Contenido
register4.v	Registro de 4 bits con reset y enable.
mux4.v	Multiplexor de entrada Y.
alu4.v	ALU de 4 bits.

El código fuente se imprime como imágenes separadas por archivo, manteniendo el formato de dos columnas del informe.

```
register4.v
/*
 * timescale 1ns/1ps
 */
# Archivos: register4.v
# Autor: Isaacel Vasquez
# Fecha: 2020-05-23
# Descripción: Registro síncrono de 4 bits con reset asincrónico y enable.
*/

module register4 (
    input wire clk,
    input wire reset,
    input wire enable,
    output reg [3:0] y
);

always @(posedge clk or posedge reset) begin
    if (reset) begin
        y <= 4'b0000;
    end else if (enable) begin
        y <= y;
    end
end

endmodule
```

Figura B1. Código fuente: register4.v.

```
Mux4.v
/*
 * timescale 1ns/1ps
 */
# Archivos: Mux4.v
# Autor: Isaacel Vasquez
# Fecha: 2020-05-23
# Descripción: Multiplexor de 4 entradas para seleccionar la entrada Y de la ALU.
*/

module mux4 (
    input wire [3:0] sel,
    input wire [3:0] a_reg,
    input wire [3:0] b_reg,
    input wire [3:0] zero,
    output reg [3:0] y
);

always @(*) begin
    case (sel)
        3'b000: y = a_reg;
        3'b001: y = b_reg;
        3'b010: y = zero;
        default: y = 4'b0000;
    end
end

endmodule
```

Figura B2. Código fuente: mux4.v.

```
alu4.v
/*
 * timescale 1ns/1ps
 */
# Archivos: alu4.v
# Autor: Isaacel Vasquez
# Fecha: 2020-05-23
# Descripción: ALU de 4 bits con suma, resta, paso de 0 y AND bit a bit.
*/

module alu4 (
    input wire [3:0] a,
    input wire [3:0] b,
    input wire [3:0] sel_alu,
    output reg [3:0] y
);

reg [4:0] temp;

always @(*) begin
    if (a == 4'b0000)
        temp = 4'b0000;
    else
        temp = {1'b0, a} + {1'b0, b};
    if (temp[4])
        temp = ~temp[4];
    end

    case (sel_alu)
        3'b000: begin
            temp = {1'b0, a} + {1'b0, b};
            y = temp[3:0];
        end
        3'b001: begin
            temp = {1'b0, a} + {1'b0, ~b};
            y = temp[3:0];
        end
        3'b010: begin
            temp = a & b;
            y = temp[3:0];
        end
        3'b011: begin
            temp = a | b;
            y = temp[3:0];
        end
        default: begin
            temp = a & b;
            y = temp[3:0];
        end
    end
end

assign y7 = (~a & ~b0000);

endmodule
```

Figura B3. Código fuente: alu4.v.

```
flag_registers.v
/*
 * timescale 1ns/1ps
 */
# Archivos: flag_registers.v
# Autor: Isaacel Vasquez
# Fecha: 2020-05-23
# Descripción: Registros de banderas C y Z con selección entre valor actual y guardado.
*/

module flag_registers (
    input wire clk,
    input wire reset,
    input wire Z,
    input wire C,
    output wire z_flag,
    output wire c_flag
);

reg z_flag_n;
reg c_flag_n;

assign c_flag = Z ? (~C & ~c_flag_n) : (~Z & c_flag_n);
assign z_flag = C ? (~C & ~z_flag_n) : (~Z & z_flag_n);

always @(posedge clk or posedge reset) begin
    if (reset) begin
        z_flag_n <= 1'b0;
        c_flag_n <= 1'b0;
    end else begin
        z_flag_n <= Z;
        c_flag_n <= C;
    end
end

endmodule
```

Figura B4. Código fuente: flag_registers.v.

flag_registers.v	Registro/selección de banderas.
datapath.v	Trayectoria de datos completa.
microstore.v	ROM de microcódigo.
control_unit.v	Unidad de control microprogramada.
proyecto_microcodificado_top.v	Módulo superior.
tb_proyecto_microcodificado.v	Banco de pruebas para ModelSim; disponible en el repositorio.

```
datapath.v
/*
 * timescale 1ns/1ps
 */
# Archivos: datapath.v
# Autor: Isaacel Vasquez
# Fecha: 2020-05-23
# Descripción: Trayectoria de datos de 4 bits con registros, MUX, ALU y banderas.
*/

module datapath (
    input wire clk,
    input wire reset,
    input wire [3:0] sel,
    input wire [3:0] a_reg,
    input wire [3:0] b_reg,
    input wire [3:0] zero,
    input wire [3:0] sel_mux,
    input wire [3:0] sel_alu,
    input wire [3:0] reg_a,
    input wire [3:0] reg_b,
    input wire [3:0] reg_c,
    input wire [3:0] reg_z,
    input wire [3:0] reg_y,
    output wire z_flag,
    output wire c_flag
);

wire [3:0] z_mux;
wire [3:0] c_mux;

mux4 mux4 (
    sel_alu,
    a_reg,
    b_reg,
    zero,
    z_mux,
    c_mux
);

alu4 alu4 (
    sel_alu,
    a_reg,
    b_reg,
    zero,
    z_mux,
    c_mux
);

register reg_a_1inst (
    clk,
    reset,
    sel_mux,
    z_mux,
    c_mux
);

register reg_b_1inst (
    clk,
    reset,
    sel_mux,
    z_mux,
    c_mux
);

register reg_c_1inst (
    clk,
    reset,
    sel_mux,
    z_mux,
    c_mux
);

flag_registers flagz (
    clk,
    reset,
    z_mux,
    c_mux,
    z_flag,
    c_flag
);

endmodule
```

Figura B5. Código fuente: datapath.v.

```
microstore.v
/*
 * timescale 1ns/1ps
 */
# Archivos: microstore.v
# Autor: Isaacel Vasquez
# Fecha: 2020-05-23
# Descripción: ROM de microcódigo para el programa de demostración.
*/

module microstore (
    input wire [3:0] address,
    output reg [3:0] instr,
    output reg [3:0] op,
    output reg [3:0] a,
    output reg [3:0] b,
    output reg [3:0] sel_mux,
    output reg [3:0] sel_alu,
    output reg [3:0] z_flag,
    output reg [3:0] c_flag
);

always @(*) begin
    instr = 4'b0000;
    op = 4'b0000;
    a = 4'b0000;
    b = 4'b0000;
    sel_mux = 4'b0000;
    sel_alu = 4'b0000;
    z_flag = 4'b0000;
    c_flag = 4'b0000;

    case (address)
        4'b0000: begin
            // Instrucción 0: 0000
            instr = 4'b0000;
            op = 4'b0000;
            a = 4'b0000;
            b = 4'b0000;
            sel_mux = 4'b0000;
            sel_alu = 4'b0000;
            z_flag = 4'b0000;
            c_flag = 4'b0000;
        end
        4'b0001: begin
            // Instrucción 1: 0001
            instr = 4'b0001;
            op = 4'b0001;
            a = 4'b0000;
            b = 4'b0000;
            sel_mux = 4'b0000;
            sel_alu = 4'b0000;
            z_flag = 4'b0000;
            c_flag = 4'b0000;
        end
        4'b0010: begin
            // Instrucción 2: 0010
            instr = 4'b0010;
            op = 4'b0010;
            a = 4'b0000;
            b = 4'b0000;
            sel_mux = 4'b0000;
            sel_alu = 4'b0000;
            z_flag = 4'b0000;
            c_flag = 4'b0000;
        end
        4'b0011: begin
            // Instrucción 3: 0011
            instr = 4'b0011;
            op = 4'b0011;
            a = 4'b0000;
            b = 4'b0000;
            sel_mux = 4'b0000;
            sel_alu = 4'b0000;
            z_flag = 4'b0000;
            c_flag = 4'b0000;
        end
        4'b0100: begin
            // Instrucción 4: 0100
            instr = 4'b0100;
            op = 4'b0100;
            a = 4'b0000;
            b = 4'b0000;
            sel_mux = 4'b0000;
            sel_alu = 4'b0000;
            z_flag = 4'b0000;
            c_flag = 4'b0000;
        end
        4'b0101: begin
            // Instrucción 5: 0101
            instr = 4'b0101;
            op = 4'b0101;
            a = 4'b0000;
            b = 4'b0000;
            sel_mux = 4'b0000;
            sel_alu = 4'b0000;
            z_flag = 4'b0000;
            c_flag = 4'b0000;
        end
        4'b0110: begin
            // Instrucción 6: 0110
            instr = 4'b0110;
            op = 4'b0110;
            a = 4'b0000;
            b = 4'b0000;
            sel_mux = 4'b0000;
            sel_alu = 4'b0000;
            z_flag = 4'b0000;
            c_flag = 4'b0000;
        end
        4'b0111: begin
            // Instrucción 7: 0111
            instr = 4'b0111;
            op = 4'b0111;
            a = 4'b0000;
            b = 4'b0000;
            sel_mux = 4'b0000;
            sel_alu = 4'b0000;
            z_flag = 4'b0000;
            c_flag = 4'b0000;
        end
        4'b1000: begin
            // Instrucción 8: 1000
            instr = 4'b1000;
            op = 4'b1000;
            a = 4'b0000;
            b = 4'b0000;
            sel_mux = 4'b0000;
            sel_alu = 4'b0000;
            z_flag = 4'b0000;
            c_flag = 4'b0000;
        end
        4'b1001: begin
            // Instrucción 9: 1001
            instr = 4'b1001;
            op = 4'b1001;
            a = 4'b0000;
            b = 4'b0000;
            sel_mux = 4'b0000;
            sel_alu = 4'b0000;
            z_flag = 4'b0000;
            c_flag = 4'b0000;
        end
        4'b1010: begin
            // Instrucción 10: 1010
            instr = 4'b1010;
            op = 4'b1010;
            a = 4'b0000;
            b = 4'b0000;
            sel_mux = 4'b0000;
            sel_alu = 4'b0000;
            z_flag = 4'b0000;
            c_flag = 4'b0000;
        end
        4'b1011: begin
            // Instrucción 11: 1011
            instr = 4'b1011;
            op = 4'b1011;
            a = 4'b0000;
            b = 4'b0000;
            sel_mux = 4'b0000;
            sel_alu = 4'b0000;
            z_flag = 4'b0000;
            c_flag = 4'b0000;
        end
        4'b1100: begin
            // Instrucción 12: 1100
            instr = 4'b1100;
            op = 4'b1100;
            a = 4'b0000;
            b = 4'b0000;
            sel_mux = 4'b0000;
            sel_alu = 4'b0000;
            z_flag = 4'b0000;
            c_flag = 4'b0000;
        end
        4'b1101: begin
            // Instrucción 13: 1101
            instr = 4'b1101;
            op = 4'b1101;
            a = 4'b0000;
            b = 4'b0000;
            sel_mux = 4'b0000;
            sel_alu = 4'b0000;
            z_flag = 4'b0000;
            c_flag = 4'b0000;
        end
        4'b1110: begin
            // Instrucción 14: 1110
            instr = 4'b1110;
            op = 4'b1110;
            a = 4'b0000;
            b = 4'b0000;
            sel_mux = 4'b0000;
            sel_alu = 4'b0000;
            z_flag = 4'b0000;
            c_flag = 4'b0000;
        end
        4'b1111: begin
            // Instrucción 15: 1111
            instr = 4'b1111;
            op = 4'b1111;
            a = 4'b0000;
            b = 4'b0000;
            sel_mux = 4'b0000;
            sel_alu = 4'b0000;
            z_flag = 4'b0000;
            c_flag = 4'b0000;
        end
    end
end

endmodule
```

Figura B6. Código fuente: microstore.v.

```
microstore.v (continuación 2)

end

default: begin
    // Instrucción no usada regresa al inicio.
    instr = 4'b0000;
    op = 4'b0000;
    a = 4'b0000;
    b = 4'b0000;
    sel_mux = 4'b0000;
    sel_alu = 4'b0000;
    z_flag = 4'b0000;
    c_flag = 4'b0000;
end

endmodule
```

Figura B7. Código fuente: *microstore.v*, continuación 2

```
control_unit_v
  include "brnch.h"
  /*
  * @branch: control_unit_v
  * @actor: instruction
  * @actor: 2000-2005
  */
  instruction: Unidad de control microprogramada con contador de dirección.
end

module control_unit {
  input wire clk;
  input wire reset;
  input wire c_fflag;
  input wire r_fflag;
  output reg in_01_address;
  output reg in_02_data;
  output reg in_03_data;
  output wire en_01;
  output wire en_02;
  output wire en_03;
  output wire [1:0] out_01;
  output wire [1:0] out_02;
  output reg [2:0] bflag;
  output reg branch_taken;
  output reg branch_taken_n;
};

wire [31:0] next_address;

microstore ram (
  address(0),data(0),
  address(1),data(1),
  address(2),data(2),
  address(3),data(3),
  address(4),data(4),
  address(5),data(5),
  address(6),data(6),
  address(7),data(7),
  address(8),data(8),
  address(9),data(9),
  address(10),data(10),
  address(11),data(11),
  address(12),data(12),
  address(13),data(13),
  address(14),data(14),
  address(15),data(15),
  address(16),data(16),
  address(17),data(17),
  address(18),data(18),
  address(19),data(19),
  address(20),data(20),
  address(21),data(21),
  address(22),data(22),
  address(23),data(23),
  address(24),data(24),
  address(25),data(25),
  address(26),data(26),
  address(27),data(27),
  address(28),data(28),
  address(29),data(29),
  address(30),data(30),
  address(31),data(31)
);

always @(clk) begin
  case (reset)
    0: begin
      2'oh0 branch_taken = 1'oh0;
      2'oh0 branch_taken_n = 1'oh0;
      2'oh0 bflag = 0'oh0;
      2'oh0 in_01_address = 0'oh0;
      2'oh0 in_02_data = 0'oh0;
      2'oh0 in_03_data = 0'oh0;
      2'oh0 out_01 = 0'oh0;
      2'oh0 out_02 = 0'oh0;
    endcase
  assign next_address = branch_taken ? next : address + 8'h0000;

  always @posedge clk or posedge reset begin
    if (~reset) begin
      2'oh0 branch_taken = 1'oh0;
      2'oh0 branch_taken_n = 0'oh0;
      2'oh0 bflag = 0'oh0;
      2'oh0 in_01_address = next_address;
      2'oh0 in_02_data = 0'oh0;
      2'oh0 in_03_data = 0'oh0;
      2'oh0 out_01 = 0'oh0;
      2'oh0 out_02 = 0'oh0;
    end
  end
endmodule
```

Figura B8. Código fuente: control unit.v

[illegible]

Figura B9. Código fuente: proyecto microcodificado top.v

[illegible]

Figura B10. Código fuente: *th* proyecto microcodificado.v

APÉNDICE C. ARCHIVO CST/UCF

En esta etapa del proyecto no se realizaron cambios a un archivo CST o UCF porque la entrega actual corresponde a la simulación en ModelSim. Por lo tanto, no existe todavía una asignación física de pines para FPGA dentro del repositorio.

Cuando el diseño se lleve a la Tang Nano 9K, este apéndice deberá actualizarse con el archivo de restricciones correspondiente, indicando las señales asignadas a reloj, reset, switches, LEDs y cualquier salida de depuración.